

Bi-Encoder vs Cross-Encoder, Worked Out by Hand

Why retrieval and re-ranking use two different architectures — one that encodes query and document *separately* for speed, one that reads them *together* for accuracy — with the cost arithmetic that forces the two-stage pipeline.

What this document does. It contrasts the bi-encoder (independent embeddings, compared by a cheap dot product) with the cross-encoder (a single model that attends over query and document jointly), counts the model invocations each needs at query time, and shows why you *retrieve* with one and *re-rank* with the other. Cost numbers in Appendix A.

This is an intuition PDF: the focus is the architectural tradeoff and its complexity, not a derivation of attention. Tuning the stage sizes is #23.

1 Two ways to score a (query, document) pair

Bi-encoder (the retriever). Encode the query and each document *independently* into vectors, then score by a cheap similarity:

$$\text{score}(q, d) = \text{enc}(q) \cdot \text{enc}(d).$$

Crucially, $\text{enc}(d)$ does not depend on q — so every document can be embedded *once*, offline, and stored in a vector index. At query time you embed only q and do fast dot products (Track 1). This is what makes searching millions of documents in milliseconds possible.

Cross-encoder (the re-ranker). Feed the query and document *together* into one transformer, let its attention layers compare every query token against every document token, and read a single relevance score off the top:

$$\text{score}(q, d) = \text{transformer}([q \parallel d]).$$

There is no reusable document vector — the computation is inseparable from q , so it must run fresh for *every* pair at query time.

Intuition

A bi-encoder is like writing a one-paragraph summary of every book in the library ahead of time, then matching a query summary against them — fast, but the summaries were written without knowing the question, so subtle relevance is lost. A cross-encoder is like handing the question *and* a full book to an expert and asking “how well does this answer it?” — far more accurate, because they read both together, but you could never do it for all million books per query. So: summaries to *shortlist*, expert to *judge the shortlist*.

2 The cost that forces a pipeline

The architectural difference is a complexity difference. For a corpus of N documents:

	Bi-encoder	Cross-encoder
doc encoding	once, offline (N total)	not possible (depends on q)
per query	1 encode + N dot products	N full transformer passes
scales to $N=10^7$?	yes (ANN, Track 1.1)	no
accuracy	good	best

A dot product is a handful of arithmetic ops; a cross-encoder pass is a full neural forward. Running the cross-encoder over all N is thousands of times more expensive per candidate *and* cannot be precomputed. Over 10^7 documents it is simply impossible within a query budget.

Mini-example • the two-stage pipeline does the math for you

Retrieve with the bi-encoder to get the top 100 candidates (fast, over all N), then run the cross-encoder on just those 100 (accurate, affordable). You pay N cheap dot products + 100 expensive passes, instead of N expensive passes. At $N = 10^7$ that is 100 cross-encoder calls rather than ten million — a $10^5 \times$ saving — while keeping cross-encoder accuracy exactly where it matters: the final ordering of the shortlist. This is *the* reason RAG retrieval is two-staged.

3 Why the cross-encoder is more accurate

The bi-encoder must compress a document into a fixed vector *before* it knows the query, so any nuance not captured in that vector is lost forever — it is a lossy summary committed too early. The cross-encoder defers: its attention lets query token “refund” interact directly with document token “reimbursement,” token “not” flip the meaning of a nearby claim, and so on. Joint reading catches negation, exact phrase alignment, and fine relevance distinctions that independent embeddings blur. The price is that this reading cannot be cached — which is exactly why it is reserved for the shortlist.

4 The division of labour

- **Bi-encoder = recall engine.** Cheap and pre-indexable: cast a wide net over all N documents to make sure the right answers are *somewhere* in the top- K .
- **Cross-encoder = precision engine.** Expensive and query-coupled: carefully re-order that top- K so the best answers reach the very top.
- **Never swap them.** A cross-encoder cannot be your first-stage retriever (it can’t scale to N); a bi-encoder alone often mis-orders the top few (it lacks joint reading). The pipeline uses each for what it is good at.

How big to make the retrieve- K and re-rank- K is the subject of PDF #23.

Equation cheat-sheet

bi: $\text{enc}(q) \cdot \text{enc}(d)$ (doc precomputed, $O(N)$ cheap),
 cross: $\text{transformer}([q||d])$ (no cache, $O(K)$ costly); retrieve bi $\rightarrow$ re-rank cross

Appendix A · Cost sketch

```
N = 10_000_000      # corpus size
K = 100              # shortlist for re-ranking

# Bi-encoder retrieval: 1 query-encode + N cheap dot products (ANN makes it ~log N)
# Cross-encoder, naive: N full transformer passes      -> infeasible
# Two-stage pipeline:   N dot products + K transformer passes
# cross-encoder calls: 100 vs 10,000,000  -> 100000x fewer, same top-K accuracy
```

Internal learning material. Cost figures are order-of-magnitude. v1.0